GovConnect — Project Summary

Overview

- Project: GovConnect (Civic Portal)
- Stack: MERN (MongoDB, Express, React, Node)
- Purpose: A minimal government portal to report crises, manage schemes and scholarships, and handle applications.

Repository layout

- `hashPassword.js` — utility script at repo root (entry referenced by package.json main).
- `package.json` — root scripts: `npm run dev` runs both server and client concurrently.
- `client/` — React frontend built with Vite.
  - `index.html` — app HTML entry.
  - `package.json` — client dependencies and scripts (`vite` dev server).
  - `vite.config.js` — vite configuration.
  - `src/` — React source:
    - `main.jsx` — React entry, mounts App.
    - `App.jsx` — top-level routes and layout.
    - `api/auth.js` — client-side wrappers for auth requests.
    - `components/` — UI components: `Navbar`, `Input`, `ProtectedRoute`, `CrisisStats`, `GovernmentStats`.
    - `pages/` — routing pages: `Home`, `Login`, `Signup`, `Dashboard`, `AdminDashboard`, `ReportCrisis`, `Schemes`, `SchemesForm`, `Scholarships`, `ScholarshipForm`, etc.
    - `utils/api.js` — generic fetch wrapper used by pages/components to call server API.
    - Styling: `index.css`, `style.css` and Tailwind/postcss configs.
- `server/` — Node + Express backend.
  - `package.json` — server dependencies (express, mongoose, jsonwebtoken, multer, dotenv, nodemon dev script).
  - `src/index.js` — server bootstrap: Express app, routes registration, MongoDB connection using `process.env.MONGO_URI` (configured in `server/.env`).
  - `src/config/db.js` — DB helper (if present) for mongoose connection.
  - `src/middleware/` — `auth.js`, `authMiddleware.js` for JWT check and role-based access.
  - `src/models/` — Mongoose models: `User`, `Scheme`, `Scholarship`, `Crisis`, `Application`.
  - `src/routes/` — Express routes organized by resource: `auth.routes.js`, `schemes.routes.js`, `scholarships.routes.js`, `crises.routes.js`, `apply.routes.js`.
  - `src/migrateItemId.js` — migration helper for existing data.
  - `uploads/` — static uploads directory served at `/uploads`.
  - `server/.env` — contains `PORT`, `MONGO_URI`, `JWT_SECRET` (sample: `mongodb://127.0.0.1:27017/civic_portal`).
- `db-backup/` — BSON backups for collections: `users.bson`, `crises.bson`, `schemes.bson`, `scholarships.bson`, `applications.bson`. Useful to restore sample data into MongoDB.

Key behaviors and flows

- Authentication:
  - `server` issues JWT tokens on login/signup via `auth.routes.js`.
  - `authMiddleware` protects routes; `adminMiddleware` gated admin-only endpoints.
- CRUD resources:
  - Schemes and Scholarships: create, read, update, delete via `schemes.routes.js` and `scholarships.routes.js`.
  - Crises: reporting via `crises.routes.js` and viewing stats on dashboard pages.
  - Applications: citizens can apply using `apply.routes.js`.
- File uploads:
  - `multer` is used in server routes to accept uploaded files; saved under `server/uploads` and served statically.

Run & development

- Prerequisites: Node.js (v16+ recommended), npm, MongoDB running locally (or provide `MONGO_URI` to Atlas).
- Start both server and client in dev mode from repo root:

```
npm install
npm run dev
```

  - Root `dev` uses `concurrently` to run server and client together.
  - Server runs on `http://localhost:5000` (default PORT in `server/.env`).
  - Client runs on `http://localhost:5173` via Vite.

Notes, potential improvements, and gotchas

- Environment: Ensure `server/.env` has a valid `MONGO_URI` and `JWT_SECRET`.
- Dependencies: Some dependencies appear both at root and subprojects; keep them in the appropriate package.json (client vs server).
- Security: `JWT_SECRET` should be strong and not committed to source.
- Scripts: Root `package.json` references `civic-portal` workspace in subprojects via `file:..` entries — those are used to symlink local packages and may be harmless, but check for duplication.
- Data restore: Use `mongorestore` to load `.bson` files from `db-backup/civic_portal`.

Files to show your friend

- README.md (README.md) — project overview and setup notes.
- `server/.env.example` — environment variable template.
- `Project_Summary.md` (this file) — concise component-by-component summary.

Generated on 2026-05-29